

ARCHITECTSVIBE · RAG FOUNDATIONS

Measuring and Improving Retrieval: Golden Datasets, Recall@K, and Advanced RAG

A learner guide to how you know your retrieval is good — and the techniques that make it better.

This guide answers two linked questions every RAG builder faces: ***how do I know my retrieval is good?*** and ***when it isn't, how do I make it better?*** We start with the yardstick (a golden dataset), then the metrics you score against it (Recall@K, MRR, nDCG, and answer-quality scoring with Ragas), then the techniques that move those scores — hybrid search and HyDE — the harder case of multi-hop questions, and finally how to record the decisions you make along the way. One idea threads through all of it:

“Retrieval quality is RAG quality.”

If the right evidence never reaches the model, no amount of clever prompting will save the answer. Everything below is about getting the right evidence in front of the model — and proving that you did.

1 · Golden datasets — your ground truth

A **golden dataset** is a curated set of high-quality examples — usually question-and-answer pairs — that acts as the benchmark for judging a model's output. It is typically built and labelled by domain experts, so it functions as the **“ground truth”**: the trusted reference your system's answers are measured against.

Think of it as the answer key for an exam. Without one, you can read your model's answers and feel they look right, but you cannot *prove* anything or compare two versions fairly. With one, evaluation becomes a repeatable measurement instead of a vibe check.

Why golden datasets matter

- **Ground-truth labelling.** Human-validated answers give you a fixed standard; any gap between the model's output and the label is a measurable error, not a matter of opinion.
- **Accuracy you can trust.** Because the examples mirror real user scenarios, scoring against them tells you whether outputs actually match what users expect.
- **Domain-specific evaluation.** You can tailor the set to your field — banking disputes, healthcare, legal — so the evaluation reflects the language and edge cases that matter to you.
- **Cost-effective and repeatable.** Building the set takes effort once; after that, you re-run it cheaply on every change instead of paying humans to review every output.
- **Benchmarking and regression-catching.** It becomes the baseline you compare new model versions and new prompts against, so a release that quietly makes things worse is caught before it ships.

What to watch when building one

- **Quality over everything.** Your evaluation is only as good as the dataset. Careful curation and cleaning matter — a biased or sloppy golden set produces confident but misleading scores.

- **Size to the use case.** A modest 10–20 examples can be enough to start; richer applications often need 100–200 diverse examples for a trustworthy picture.
- **Diversity is the hard part.** Cover varied phrasings, demographics, and contexts so the score reflects the real spread of inputs — not just the easy ones. This is resource-intensive but essential to avoid hidden blind spots.

Where this leads: once you have a golden dataset of questions plus the documents that *should* be retrieved for each, you can put a number on retrieval quality. That number is **Recall@K**.

2 · Scoring your retrieval — Recall@K, MRR & nDCG

Once you have a golden dataset, you can put numbers on retrieval quality. Three metrics answer three different questions — they are best used together.

2.1 · Recall@K — did we find everything?

Recall@K (“Recall at K”) is an evaluation metric from search and recommendation systems. It measures the share of the **truly relevant** items your system manages to retrieve within its top K results, out of all the relevant items that exist for that query.

The formula is simply a ratio:

$$\text{Recall@K} = \frac{\text{Number of relevant items in the top K}}{\text{Total number of relevant items that exist}}$$

A worked example

Suppose you are testing a search system over a database that contains **10** relevant articles about “Quantum Physics.” You ask for the top $K = 5$ results, and **3** of those five turn out to be genuinely relevant articles. Then:

$$\text{Recall@5} = 3 / 10 = 0.3 \quad (30\%)$$

In words: by looking only at the top five results, the system surfaced **30% of all the relevant information** that was available. The other 70% existed in the database but did not make it into the top five.

Why it matters

- **It measures comprehensiveness.** Recall@K tells you whether your system is capturing the relevant material or quietly missing it. A low score means good evidence exists but your retriever isn’t finding it.
- **K is a dial you can turn.** By checking Recall@1, Recall@5, and Recall@10, you see how quickly relevance arrives. A high Recall@5 means users rarely have to scroll far before they hit what they need.

2.2 · MRR — how high is the first hit?

Mean Reciprocal Rank (MRR) asks a sharper question: when there is a correct result, **how near the top is the first one** — averaged across all your test queries. For each query you take the *reciprocal rank* of the first relevant result: position 1 scores $1/1 = 1.0$, position 2 scores $1/2 = 0.5$, position 5 scores $1/5 = 0.2$, and a query with no hit at all scores 0. MRR is the average of those across every query.

Example: across three queries the first relevant result lands at positions 1, 3 and 2. The reciprocal ranks are 1.0, 0.33, 0.5, so $MRR = (1.0 + 0.33 + 0.5) / 3 = 0.61$.

Use it when the user mainly needs **one good answer, fast** — a single best document rather than an exhaustive set. Recall@K asks “did we find them all?”; MRR asks “how high was the first one?”

2.3 · nDCG — is the whole ranking well-ordered?

Normalised Discounted Cumulative Gain (nDCG) is the most complete of the three: a **rank-aware quality score** that rewards putting relevant results near the top and *discounts* relevant results that sit lower down. It can also use graded relevance (“highly” vs “somewhat” relevant), applies a logarithmic position discount, then *normalises* against the best possible ordering so the score lands between 0 and 1 — where 1.0 is a perfect ranking.

In plain terms: “discounted” means lower positions count for less, and “normalised” means you are graded against the ideal order rather than in the abstract. Reach for nDCG when the **order of several relevant results** matters, not just whether they appear at all.

Which one when:

Metric	Question it answers	Reach for it when...
Recall@K	Did we find all the relevant items in the top K?	Completeness matters — you need every relevant document.
MRR	How high is the first correct result?	One good answer, fast, is what the user needs.
nDCG	Is the whole ranked list well-ordered?	The order of multiple relevant results matters.

How this connects back

Recall@K is only meaningful *because* of the golden dataset: the “total number of relevant items” in the denominator comes from your expert-labelled ground truth. No golden set, no honest denominator — and no trustworthy score.

3 · Scoring the answer, not just retrieval — Ragas

Recall@K, MRR and nDCG all judge **retrieval** — did we fetch the right documents? But RAG has a second half: the answer the LLM actually writes. **Ragas** is a widely-used open-source library that scores that answer quality automatically (often using an LLM as the judge), so you can evaluate the generation step at scale. Three of its core measures:

- **Faithfulness.** Is every claim in the answer actually supported by the retrieved context? This is the anti-hallucination check — a confident answer that invents facts scores low.
- **Answer relevancy.** Does the answer actually address the question that was asked, without padding or drifting off-topic?
- **Context precision.** Of the chunks you retrieved and fed the model, how many were genuinely relevant? Low precision means good context is drowning in noise — which also wastes tokens and can confuse the model.

The full evaluation picture: retrieval metrics (Recall@K, MRR, nDCG) tell you whether the right evidence was *found*; Ragas tells you whether the model then *used it well*. You want both green.

4 · Improving the score — advanced retrieval

Suppose your Recall@K is disappointing. The usual culprit is the retriever itself. A plain vector (semantic) search is powerful but has two well-known weak spots: it can **miss exact keywords** (product IDs, acronyms, precise jargon), and it can **struggle with short, ambiguous queries** where there simply isn't enough text to embed well. The two techniques below target exactly those gaps.

4.1 · Hybrid search: BM25 + RRF + a re-ranker

This is the industry-standard “hybrid search” pattern for pulling the most precise context for your model. It combines three pieces:

- **BM25 — the keyword specialist.** A traditional, sparse keyword-matching algorithm — think “Ctrl-F on steroids.” It excels at finding exact product IDs, acronyms, and specific technical jargon that vector embeddings tend to blur over.
- **RRF (Reciprocal Rank Fusion) — the fair referee.** BM25 and vector search score on completely different scales (BM25 produces unbounded numbers; vector similarity sits between 0.0 and 1.0). RRF sidesteps the mismatch by ignoring the raw scores and looking only at each document's *rank position* in each list. Documents that rank high in *both* lists get the highest fused score — no fragile normalisation required.
- **Cross-Encoder re-ranker — the final judge.** Usually the last step. A model that reads each (query, document) pair together and deeply re-scores them, re-ordering the fused list so the single most relevant answers rise to the very top.

Mental model: BM25 catches the exact words, vector search catches the meaning, RRF fairly merges the two opinions, and the cross-encoder does the careful final read. Each step covers a weakness of the others.

4.2 · HyDE (Hypothetical Document Embeddings)

HyDE is a query-rewriting technique that fixes a subtle mismatch: humans ask questions casually and briefly, but the documents we search are written as dense, declarative answers. Embedding a short question and comparing it to long answers is an apples-to-oranges match. HyDE makes it apples-to-apples.

Instead of embedding the raw user query, HyDE does three things:

1. **Generate.** Pass the query to an LLM and ask it to write a detailed, *hypothetical* answer to the question.
2. **Embed.** Convert that synthesised answer into a dense vector.
3. **Retrieve.** Use that hypothetical document's vector to search your knowledge base.

Why it works: it forces the search to match **document-to-document** (the synthetic answer against the real documents) rather than question-to-document. The hypothetical text acts as rich context, expanding the query's vocabulary and surfacing evidence-heavy chunks instead of shallow keyword matches — which is exactly why it rescues short or poorly-worded questions.

5 · A harder case — multi-hop questions

Some questions can't be answered from a single document. A **multi-hop** question needs you to combine information from **two or more** documents — each retrieval is a “hop.” For example: “Which dispute-resolution SLA applies to the product the customer in ticket #4821 bought?” First you must find the product (one document), then the SLA for that product (another).

Why it is hard: a single search retrieves documents similar to the original query, but **no one document holds the whole answer**, and the second document you need may look unrelated to the question as asked. Single-shot retrieval often misses it.

What helps: **query decomposition** (break the question into sub-questions and retrieve for each) and **iterative or agentic retrieval** (retrieve, read, form a follow-up query, retrieve again). When you evaluate multi-hop, your golden dataset must record the full *chain* of documents required, not just a single “correct” passage.

6 · How it all fits together

In a production-grade RAG system you rarely pick just one of these — you **chain** them. A user's query is first expanded by HyDE, retrieved in parallel by BM25 and vector search, fused by RRF, sharpened by a cross-encoder, and only then handed to the LLM as context:

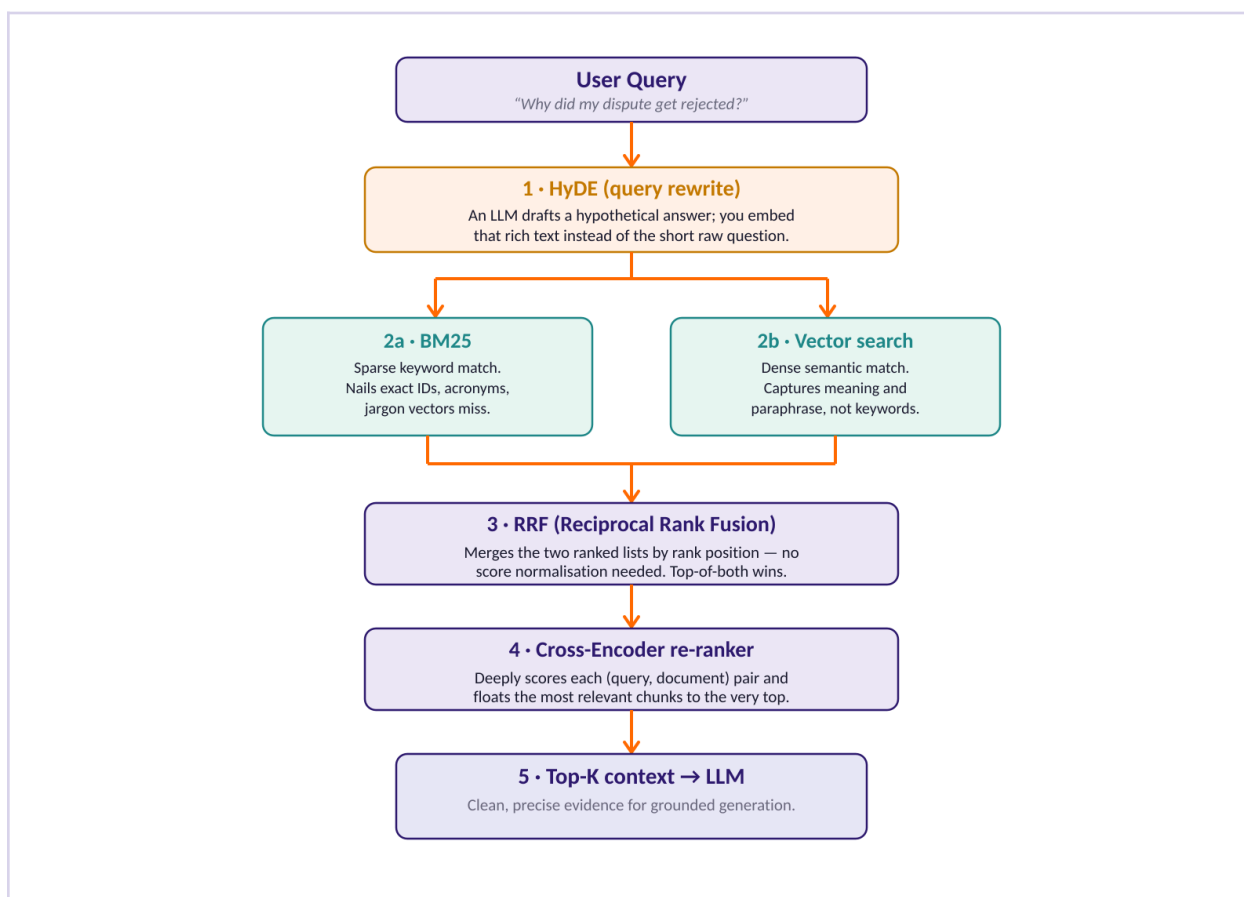


Figure 1 — The production RAG retrieval pipeline: each stage covers a weakness of the one before it.

Read end to end, the flow is: **User Query** → **HyDE** → **BM25 & Vector (parallel)** → **RRF** → **Cross-Encoder** → **Context** → **LLM**. And the way you *know* each change helped? You re-run your golden

dataset and watch Recall@K go up. That is the whole loop: measure with a golden set, improve with better retrieval, measure again.

7 · Capturing your decisions — ADRs

Tuning all of this is a stream of decisions: hybrid search or pure vector? Which value of K? Which re-ranker model? HyDE on or off? Each is a trade-off that a teammate — or future-you — will question months later. An **Architecture Decision Record (ADR)** is a short, dated note that captures *one* such decision and the reasoning behind it, so the team never has to relitigate or guess why a choice was made.

A typical ADR is just a few short sections, kept as a small Markdown file in the repo (for example `docs/adr/0007-use-hybrid-search.md`):

- **Title & status.** What was decided, and whether it is proposed, accepted, or superseded.
- **Context.** The forces at play — e.g. “pure vector search scored Recall@5 = 0.42 on jargon-heavy dispute queries.”
- **Decision.** The choice made — e.g. “adopt BM25 + vector + RRF with a cross-encoder re-ranker.”
- **Consequences.** What this makes easier or harder — better recall on exact terms, at the cost of a heavier pipeline and a re-ranker latency budget.

The link to everything above: the metrics are your *evidence*; the ADR is where you write down what that evidence made you decide. Grounded decisions, not folklore.

Quick recap

- **Golden dataset** — expert-labelled ground truth; your answer key and benchmark.
- **Recall@K / MRR / nDCG** — retrieval scorecards: did we find them all / how high is the first hit / is the whole ranking well-ordered.
- **Ragas** — scores the generated answer: faithfulness, answer relevancy, context precision.
- **BM25 + RRF + re-ranker** — hybrid search: exact keywords, fair fusion, careful final ranking.
- **HyDE** — rewrite the query into a hypothetical answer so search matches document-to-document.
- **Multi-hop** — answers needing 2+ documents; use query decomposition or iterative retrieval.
- **ADR** — a short, dated record of each architecture decision and why you made it.
- **The loop** — measure (golden set + metrics + Ragas) → improve (hybrid + HyDE) → record (ADR) → measure again.

Further reading

To take these from concept to implementation:

- **Hybrid search with RRF** — implementation guides in the Qdrant documentation and the Weaviate blog (“Hybrid Search Explained”).
- **HyDE** — the foundational concepts in the LangChain guides, and the original zero-shot dense retrieval paper on arXiv ([arXiv:2212.10496](https://arxiv.org/abs/2212.10496)).

- **Re-rankers** — search for “cross-encoder re-ranking” in the Sentence-Transformers documentation to see the (query, document) scoring pattern in code.
- **Evaluation & Ragas** — the Ragas documentation covers faithfulness, answer relevancy and context precision; for the metric formulas, look up “MRR” and “nDCG” in any information-retrieval reference.